

GestSudo

Documentación para Desarrolladores

Empresa: Sudo Agile S.A.

Software: GestSudo

Fundador: Eduardo Ramirez De Lama

Challenge Alura Agente

Documento corporativo oficial

1. Control de versiones del documento

Versión	Fecha	Responsable	Rol	Descripción de cambios
2.0	2026-07-19	Eduardo Ramirez De Lama	Fundador / Product Owner	Reestructuración corporativa, identidad GestSudo e integración completa del módulo IA.

Nota: mantener este cuadro actualizado en cada revisión formal del documento.

2. Tabla de contenido

1. Control de versiones del documento
2. Tabla de contenido
3. Resumen Ejecutivo
4. Introducción
5. Objetivos
6. Arquitectura general del sistema
7. Modelo de dominio funcional
8. Backend
9. Frontend
10. Seguridad y autenticación
11. API REST y catálogo de endpoints
12. Flujos de negocio relevantes
13. Configuración y ambientes
14. Ejecución local y runbook operativo
15. Pruebas y validación
16. Buenas prácticas y convenciones
17. Módulo de Inteligencia Artificial
18. Roadmap técnico

- 19. Glosario técnico
- 20. Checklist de continuidad
- 21. Errores comunes y resolución
- 22. FAQ técnica
- 23. Conclusiones

3. Resumen Ejecutivo

GestSudo es un software empresarial desarrollado por Sudo Agile S.A. para la gestión ágil de proyectos, módulos, tareas y subtarear con control por roles y trazabilidad operativa.

Dentro de GestSudo, el Challenge Alura Agente se implementa como un módulo de Inteligencia Artificial integrado que permite consultar documentación empresarial en lenguaje natural mediante arquitectura Retrieval-Augmented Generation (RAG).

Esta capacidad fortalece el acceso a conocimiento institucional y acelera la toma de decisiones sin alterar la arquitectura funcional principal de la plataforma.

Empresa propietaria: Sudo Agile S.A.

Software: GestSudo

Fundador: Eduardo Ramirez De Lama

4. Introducción

Este documento consolida la información técnica y funcional de GestSudo para equipos de desarrollo, continuidad operativa y gobierno tecnológico. El módulo de Inteligencia Artificial se documenta como componente integrado de la plataforma y no como producto independiente.

5. Objetivos

1. Mantener una base documental completa para operación, mantenimiento y evolución de GestSudo.
2. Unificar criterios técnicos de arquitectura, seguridad, API y despliegue.
3. Preservar trazabilidad funcional para soporte técnico y continuidad del negocio.
4. Definir lineamientos de integración del módulo de IA alineado con el Challenge Alura Agente.

6. Arquitectura general del sistema

GestSudo implementa una arquitectura full stack orientada a gestión ágil empresarial con jerarquía funcional:

- Proyecto → Módulo → Tarea → Subtarea

Capacidades clave:

- Autenticación JWT y control de acceso por roles.
- CRUD y operaciones de flujo por entidad de negocio.
- Asignación de usuarios y prioridades por nivel jerárquico.
- Gestión de adjuntos por tipo de artefacto.
- Archivado y eliminación lógica de proyectos.
- Exportación de proyectos a Excel.

7. Modelo de dominio funcional

Categoría	Elementos
Entidades nucleares	Proyecto, Módulo, Tarea, Subtarea, Usuario, Prioridad, Área Funcional
Entidades de soporte	Flujos por nivel, adjuntos por tipo, relaciones usuario-rol y asignaciones
Roles	ADMINISTRADOR, GESTOR, DESARROLLADOR, DISEÑADOR, QA, DEVOPS, SOPORTE

Estados oficiales por entidad:

Entidad	Estados
Proyecto	PENDIENTE, EN_PROGRESO, COMPLETADO, ENTREGADO, ARCHIVADO
Módulo	PENDIENTE, EN_PROGRESO, COMPLETADO, CANCELADO
Tarea	PENDIENTE, EN_PROGRESO, COMPLETADA, CANCELADO
Subtarea	PENDIENTE, EN_PROGRESO, COMPLETADA, CANCELADO

8. Backend

- Stack: Java 21, Spring Boot 3.1.9, Spring Security, Spring Data JPA/Hibernate, SQL Server JDBC, JWT, OpenFeign, Apache POI.
- Estructura por capas: controller → service → repository.
- Main class: com.codigo.msregistro.application.ApplicationLauncher.
- Módulos principales del build Maven: application, domain, infraestructure.

9. Frontend

- Stack: React 17, Vite, Ant Design, Axios, react-router-dom, SweetAlert2, chart.js.
- Archivo principal: gestor-frontend/src/App.jsx.
- Rutas de alto impacto: /proyectoList, /proyectoListArchivados, /proyectoPapelera, /usuarioslista, /areas-funcionales, rutas de módulos/tareas/subtareas.
- Convención de sesión: decodificación de JWT en cliente y cierre de sesión por expiración.
- Observación técnica: coexistencia de estrategias de consumo HTTP (instance y llamadas directas).

10. Seguridad y autenticación

Archivo de referencia: SecurityConfiguration.java

- Seguridad stateless con SessionCreationPolicy.STATELESS.
- Filtro JWT previo a UsernamePasswordAuthenticationFilter.
- Rutas /api/v1/autenticacion/** públicas para inicio de sesión y registro.
- Autorización por autoridad/rol en rutas de negocio.
- CORS habilitado con orígenes de desarrollo permitidos (localhost:5173 y 127.0.0.1:5173).

Advertencia: no exponer secretos ni credenciales en código ni repositorios.

11. API REST y catálogo de endpoints

Nota: conservar este catálogo como referencia funcional y técnica para desarrollo e integración.

11.1 Autenticación

Grupo	Endpoints principales
Base	/api/v1/autenticacion
Operaciones	signin, signupuser, signupadmin, validateToken, claveToken

11.2 Usuarios, roles y áreas

Grupo	Endpoints principales
Usuarios	/api/usuarios, /api/usuarios/getCurrentUser, /api/usuarios/{id}, /api/usuarios/{id}/estado
Roles	/api/usuariosrol/todos, /api/usuariosrol/{id}/roles, /api/usuariosrol/roles, /api/rol
Áreas funcionales	/api/areas-funcionales (GET, POST)
Prioridades	/api/prioridad (GET, POST)

11.3 Proyectos

Grupo	Endpoints principales
Base	/api/proyectos

Operaciones	GET listado/detalle, POST creación, PATCH actualización y fechas, PUT estado/posición/usuarios/prioridad, archivado, papelera, restauración, descarga Excel
Rutas destacadas	/api/proyectos/no-archivados, /api/proyectos/archivados, /api/proyectos/papelera, /api/proyectos/descargar

11.4 Módulos

Grupo	Endpoints principales
Base	/api/proyectosmodulo/{proyectold}/modulos
Operaciones	creación, edición, fechas inicio/fin, estado, posición, asignación de usuarios y prioridad, eliminación

11.5 Tareas

Grupo	Endpoints principales
Base	/api/modulos/{modulold}/tareas
Operaciones	creación, edición, fechas, estado, posición, usuarios, prioridad, eliminación

11.6 Subtareas

Grupo	Endpoints principales
Base	/api/tareas/{tareald}/subTareas
Operaciones	creación, edición, fechas, estado, posición, usuarios, prioridad, eliminación

11.7 Flujos y adjuntos

Grupo	Endpoints principales
Flujos	/api/flujos/proyectos/*, /api/flujos/modulos/*, /api/flujos/tareas/*, /api/flujos/subtareas/*
Adjuntos	/api/aplicativos, /api/marcos, /api/diccionarios, /api/documentos, /api/productos, /api/basesdato por nivel
Patrones	mostrar/{entidadId}/{usuariold}, update/descripcion, update/progreso, update/fechaA, GET todo/{id}, POST multipart, descarga por id

12. Flujos de negocio relevantes

5. Ciclo principal: crear proyecto → módulos → tareas → subtareas → asignaciones → prioridad/estado/fechas → seguimiento por flujos.

6. Cierre de proyecto: COMPLETADO/ENTREGADO, ARCHIVADO o eliminación lógica con recuperación.
7. Gobierno de permisos: control por rol/authority en backend y visibilidad condicional en frontend.

13. Configuración y ambientes

- Archivos de configuración: application.properties, application-local.properties, application-cloud.properties.
- Lineamiento: externalizar secretos por ambiente.
- No hardcodear tokens, firmas ni credenciales.
- Parametrizar URLs de backend/frontend por entorno.

Control	Validación
Datasource	URL, usuario y contraseña correctos por ambiente
Seguridad	key.signature aislada por entorno
CORS	orígenes válidos para entorno activo
Integración	coherencia de URL frontend/backend

14. Ejecución local y runbook operativo

Prerequisitos: Java 21, Maven 3.9+, Node.js LTS, npm y SQL Server accesible.

Backend:

```
cd C:\SudoProjects\SudoGes\SudoAgile-main
mvn clean install
mvn -pl application spring-boot:run
```

Frontend:

```
cd C:\SudoProjects\SudoGes\SudoAgile-main\gestor-frontend
npm install
npm run dev
```

Puertos habituales: API 8080, Frontend 5173.

15. Pruebas y validación

- Inicio/cierre de sesión con múltiples roles.
- CRUD completo de proyecto, módulo, tarea y subtarea.
- Asignación y desasignación de usuarios.
- Cambio de prioridad, estado y fechas.
- Archivado, restauración y papelera.
- Flujos (descripción, progreso y fecha ampliada).

- Carga y descarga de adjuntos.
- Build de backend y frontend sin errores.

16. Buenas prácticas y convenciones

- Estandarizar contratos de error (code, message, details).
- Unificar el cliente HTTP del frontend para reducir inconsistencias.
- Evitar lógica de negocio en controladores.
- Separar DTOs y entidades cuando aplique.
- Normalizar nomenclatura de rutas y payloads.
- Mantener consistencia idiomática y de naming en el código.

17. Módulo de Inteligencia Artificial

Objetivo del módulo

El módulo de IA de GestSudo proporciona respuestas en lenguaje natural sobre documentación empresarial. Está integrado en la plataforma y no constituye un producto independiente.

Arquitectura RAG

- Ingesta de PDFs desde carpeta docs.
- Segmentación en chunks con solapamiento.
- Vectorización con text-embedding-3-small (Azure OpenAI).
- Persistencia de índice FAISS en db/faiss_index.
- Recuperación semántica de contexto relevante.
- Generación de respuesta con GPT-5-mini en Azure OpenAI.
- Interfaz conversacional Streamlit para usuarios internos.

Componente	Detalle
Azure OpenAI	Proveedor de modelos para chat y embeddings
GPT-5-mini	Modelo de generación de respuestas
text-embedding-3-small	Modelo de embeddings para indexación documental
LangChain	Orquestación del pipeline RAG y construcción de prompts
FAISS	Motor de búsqueda vectorial local
Streamlit	Canal de interacción de usuario para el asistente

Flujo de recuperación documental

8. Usuario formula consulta en la interfaz.
9. La consulta se transforma a embedding.

10. FAISS recupera fragmentos relevantes.
11. LangChain compone prompt con contexto y reglas.
12. GPT-5-mini genera respuesta final con soporte documental.

Construcción del índice vectorial

Comando operativo: `py -3 app.py`

Nota: regenerar índice cuando se modifiquen documentos fuente.

Variable de entorno	Uso
AZURE_OPENAI_ENDPOINT	Endpoint del servicio Azure OpenAI
AZURE_OPENAI_API_KEY	Clave de autenticación
AZURE_OPENAI_CHAT_DEPLOYMENT	Deployment de GPT-5-mini
AZURE_OPENAI_EMBEDDING_DEPLOYMENT	Deployment de text-embedding-3-small
AZURE_OPENAI_CHAT_API_VERSION	Versión API de chat
AZURE_OPENAI_EMBEDDING_API_VERSION	Versión API de embeddings

Integración con GestSudo

- El asistente IA se integra como módulo funcional dentro de GestSudo.
- Comparte lineamientos de seguridad, operación y gobierno documental de la plataforma.
- Su evolución se alinea al roadmap técnico corporativo.

18. Roadmap técnico

13. Corto plazo: unificación de capa HTTP frontend y normalización de errores API.
14. Mediano plazo: especificación OpenAPI y suite automatizada por capas.
15. Largo plazo: observabilidad avanzada, modularización de frontend por dominio y gestión de secretos vía vault.

19. Glosario técnico

RAG: Retrieval-Augmented Generation para respuestas con contexto documental.

JWT: Token de autenticación firmado para control de sesiones.

FAISS: Índice vectorial para búsqueda semántica eficiente.

Flujo: Registro de avance por entidad (descripción/progreso/fecha).

Papelera: Eliminación lógica con posibilidad de recuperación.

20. Checklist de continuidad

[] Validar salud de autenticación y permisos por rol.

- [] Verificar coherencia de endpoints consumidos por frontend.
- [] Revisar cargas de adjuntos y procesos de descarga.
- [] Confirmar integridad de estados y flujos por entidad.
- [] Actualizar documentación tras cada cambio de arquitectura o API.

21. Errores comunes y resolución

Caso	Causa probable	Acción recomendada
401/403 recurrente	Token expirado o rol sin autoridad	Revalidar sesión, revisar SecurityConfiguration y endpoint invocado
Error CORS en desarrollo	Origen no permitido	Verificar CorsConfig y URL/proxy de frontend
No se visualizan datos	Filtro por rol/gestor	Consultar usuario actual y filtros activos
Falla en adjuntos	Endpoint o payload multipart incorrecto	Validar tipo de artefacto, permisos y formato de envío

22. FAQ técnica

P: ¿Puede el módulo IA operar sin documentos indexados?

R: No. Requiere índice FAISS construido con documentación válida.

P: ¿El asistente IA reemplaza la API REST?

R: No. Es un módulo de consulta documental integrado en GestSudo.

P: ¿Dónde se controla el acceso por rol?

R: En SecurityConfiguration del backend y en validaciones de visibilidad frontend.

P: ¿Qué hacer ante respuestas ambiguas de IA?

R: Refinar la consulta, revisar documentos fuente y regenerar índice si hubo cambios.

23. Conclusiones

GestSudo dispone de una base técnica sólida para operación empresarial y evolución controlada. La incorporación del módulo de Inteligencia Artificial fortalece el acceso al conocimiento documental interno y se integra de forma coherente con la arquitectura, seguridad y roadmap de la plataforma.

SudoGes — Alcance de Base de Datos (Addendum cap. 24-27)

Nota de actualizacion: el nombre del software actualiza de GestSudo a SudoGes en toda la documentacion.

24. Alcance completo de la Base de Datos — Motor y estrategia

Motor principal: Microsoft SQL Server (local: localhost:1433/ges, cloud: gesinei.lat:1433/gesSudo).

ORM: Hibernate (Spring Data JPA). Estrategia: ddl-auto=update.

Zona horaria configurada: America/Lima.

Tamano maximo de adjunto: 100 MB por archivo.

TABLAS MAESTRAS Y DE SEGURIDAD

- 1) usuarios — Datos personales, email (unique), password (bcrypt), telefono, genero, nacimiento, fecha_registro, background_user, activo, id_superior (FK self-referencing), relacion con rol.
- 2) rol — Catalogo de roles: id_rol, nombre_rol (unique).
- 3) usuario_rol — Tabla puente N:M usuarios <-> roles.
- 4) area_funcional — id, nombre, id_gestor (FK usuarios), creado_por (FK usuarios).
- 5) prioridad — id, nombre, background_prioridad (color hex 7 chars).

25. Tablas operativas principales

- 6) proyectos — id, nombre, descripcion, estado (EstadoProyecto), fecha_inicio, fecha_fin, fecha_ampliada, background_proyecto, id_proyecto_orden, eliminado (bool), archivar_at, archivar_delete, create_at, user_create, modify_at, user_modify, delete_at, user_delete, FK: prioridad_id, id_area_funcional.
- 7) modulos — id, nombre, descripcion, estado (EstadoModulo), fecha_inicio, fecha_fin, id_modulo_orden, create_at, user_create, modify_at, user_modify, delete_at, user_delete, FK: proyecto_id, prioridad_id.
- 8) tareas — id, nombre, descripcion, estado (EstadoTarea), fecha_inicio, fecha_fin, id_tarea_orden, create_at, user_create, modify_at, user_modify, delete_at, user_delete, FK: modulo_id, prioridad_id.
- 9) subtareas — id, nombre, descripcion, estado (EstadoSubtarea), fecha_inicio, fecha_fin, id_subtarea_orden, create_at, user_create, modify_at, user_modify, delete_at, user_delete, FK: tarea_id, prioridad_id.

26. Tablas de asignacion de responsables (N:M)

- 10) proyecto_usuario — proyecto_id <-> usuario_id
- 11) modulo_usuario — modulo_id <-> usuario_id
- 12) tarea_usuario — tarea_id <-> usuario_id
- 13) subtarea_usuario — subtarea_id <-> usuario_id

27. Tablas de flujo y seguimiento de avance

Columnas comunes: id, estado_actual, porcentaje_avance, descripcion, fecha_ampliada,

FK: {entidad}_id, usuario_id.

14) flujos_proyecto

15) flujos_modulo

- 16) flujos_tarea
- 17) flujos_subtarea

SudoGes — Base de Datos: Adjuntos, Puentes y Recomendaciones (cap. 28-31)

28. Tablas de adjuntos por nivel (artefactos documentales)

Estructura comun de todas las tablas de adjuntos:

id, nombre_archivo, nombre_archivo_original, tipo_archivo,
tamano_archivo, ruta_archivo, usuario_id, fecha_registro.

NIVEL PROYECTO

- 18) aplicativos_proyecto
- 19) marcos_proyecto
- 20) diccionarios_proyecto
- 21) documentos_proyecto
- 22) productos_proyecto
- 23) bases_dato_proyecto

NIVEL MODULO

- 24) aplicativos_modulo
- 25) marcos_modulo
- 26) diccionarios_modulo
- 27) documentos_modulo
- 28) productos_modulo
- 29) bases_dato_modulo

NIVEL TAREA

- 30) aplicativos_tarea

- 31) marcos_tarea

NIVEL SUBTAREA

- 32) aplicativos_subtarea

- 33) marcos_subtarea

29. Tablas puente — asociacion entre entidades y adjuntos (N:M)

PROYECTO <-> adjuntos

- 34) moduloaplicativo_proyecto — proyecto_id <-> aplicativo_id
- 35) modulomarco_proyecto — proyecto_id <-> marco_id
- 36) modulodiccionario_proyecto — proyecto_id <-> diccionario_id
- 37) modulodocumento_proyecto — proyecto_id <-> documento_id
- 38) moduloproducto_proyecto — proyecto_id <-> producto_id
- 39) modulobasesdato_proyecto — proyecto_id <-> basesdato_id

MODULO <-> adjuntos

- 40) moduloaplicativo_modulo — modulo_id <-> aplicativo_id
- 41) modulomarco_modulo — modulo_id <-> marco_id
- 42) modulodiccionario_modulo — modulo_id <-> diccionario_id
- 43) modulodocumento_modulo — modulo_id <-> documento_id
- 44) moduloproducto_modulo — modulo_id <-> producto_id
- 45) modulobasesdato_modulo — modulo_id <-> basesdato_id

TAREA <-> adjuntos

- 46) taeraaplicativo_tarea — tarea_id <-> aplicativo_id (nota: typo en nombre original)
- 47) tareamarco_tarea — tarea_id <-> marco_id

SUBTAREA <-> adjuntos

- 48) subtareaaplicativo_tarea — subtarea_id <-> aplicativo_id

49) subtareamarco_subtarea — subtarea_id <-> marco_id

Total de tablas identificadas en el modelo JPA actual: 49.

30. Enums de estado y campos de auditoria

ENUMS (persistidos como STRING):

EstadoProyecto: PENDIENTE, EN_PROGRESO, COMPLETADO, ENTREGADO, ARCHIVADO.

EstadoModulo: PENDIENTE, EN_PROGRESO, COMPLETADO, CANCELADO.

EstadoTarea: PENDIENTE, EN_PROGRESO, COMPLETADA, CANCELADO.

EstadoSubtarea: PENDIENTE, EN_PROGRESO, COMPLETADA, CANCELADO.

PATRON DE AUDITORIA (aplicado en proyectos, modulos, tareas, subtareas):

create_at DATE/TIMESTAMP — fecha de creacion.

user_create VARCHAR — usuario que creo el registro.

modify_at DATE/TIMESTAMP — fecha de ultima modificacion.

user_modify VARCHAR — usuario que modifiko.

delete_at DATE/TIMESTAMP — fecha de eliminacion logica.

user_delete VARCHAR — usuario que elimino.

31. Recomendaciones tecnicas de base de datos

- Implementar versionado de esquema con Flyway o Liquibase en entornos productivos.
- Definir indices en: estado, prioridad_id, fecha_inicio, fecha_fin, FK principales.
- Corregir typos en nombres de tablas puente (ej. taeraaplicativo_tarea) en proxima migracion.
- Unificar convencion singular/plural y prefijos en tablas de relacion.
- Revisar nombre de BD en cloud (gesSudo) para alinear con nombre oficial SudoGes.
- Establecer politica formal de backup y retencion de archivos adjuntos.
- Separar almacenamiento de binarios a servicio dedicado (S3/Azure Blob) en produccion.

SudoGes — Exportacion a Excel: implementacion tecnica (cap. 32)

32. Exportacion a Excel — Implementacion tecnica

El sistema implementa exportacion de proyectos a Excel mediante Apache POI (XSSFWorkbook).

STACK TECNICO:

Libreria : Apache POI (incluida en pom.xml como dependencia Spring Boot).
Formato : XLSX (Office Open XML) — XSSFWorkbook.
Endpoint : GET /api/proyectos/descargar
Controller: ProyectoController.java -> metodo exportarProyectos()
Service : ProyectoService.java -> metodo exportarProyectosAExcel(List<Proyecto>)
Frontend : proyectoList.jsx -> funcion descargarProyectosExcel()
Service FE: projectService.js -> funcion descargarProyectosExcels(token)

LOGICA DEL ENDPOINT (ProyectoController):

1. Obtiene el usuario autenticado via SecurityContextHolder.
2. Determina el conjunto de proyectos segun rol:
 - ADMINISTRADOR : listarProyectosNoArchivados() (todos).
 - GESTOR : listarProyectosNoArchivadosPorGestor(usuario.getId()).
 - Otros : listarProyectosNoArchivadosPorGestor(usuario.getSuperior().getId()).
3. Invoca exportarProyectosAExcel(proyectos) -> ByteArrayInputStream.
4. Retorna ResponseEntity con Content-Type XLSX y header:
Content-Disposition: attachment; filename=proyectos.xlsx

ESTRUCTURA DEL EXCEL GENERADO:

Fila 0 : Titulo "Lista de Proyectos" con merge de celdas y fuente Bold 14pt.
Fila 2 : Cabeceras con fondo BLUE_GREY, texto blanco Bold 11pt, bordes THIN.
Fila 3+ : Datos con bordes THIN y alineacion vertical CENTER.

COLUMNAS GENERADAS (19 columnas):

Proyecto, Descripcion, Fecha.I P, Fecha.F P, Estado P,
Modulo, Fecha.I M, Fecha.F M, Estado M,
Tarea, Fecha.I T, Fecha.F T, Estado T,
Subtarea, Estado ST, Fecha.I ST, Fecha.F ST,
Usuario Creador, Fecha Creacion.

MANEJO DE JERARQUIA INCOMPLETA:

- Proyecto sin modulos : fila con "Sin modulo", "Sin tarea", "Sin subtarea".
- Modulo sin tareas : fila con nombre de modulo y "Sin tarea", "Sin subtarea".
- Tarea sin subtareas : fila con nombre de tarea y "Sin subtarea".

LLAMADA DESDE FRONTEND (projectService.js):

instance.get("/proyectos/descargar", { responseType: "blob" })
-> Crea URL.createObjectURL(blob) -> dispara descarga automatica en el navegador.

NOTA DE SEGURIDAD: el endpoint respeta el filtro JWT y la politica de roles.

No es necesario parametro adicional; el backend infiere el alcance desde el token.

33. Archivos del codigo fuente utilizados para la programacion de Tareas

Esta seccion detalla cada archivo del proyecto involucrado en la logica de tareas, organizado por capa arquitectonica (dominio, infraestructura, aplicacion y frontend).

CAPA DE DOMINIO — domain/src/main/java/.../aggregates/

Tarea.java

Entidad JPA principal de la tarea. Define todos los campos persistibles: nombre, descripcion, estado (EstadoTarea), fecha_inicio, fecha_fin, id_tarea_orden, prioridad (FK), modulo (FK), usuarios (N:M), subtareas (OneToMany), marcos (N:M), aplicativos (N:M), auditoria.
Tabla: tareas.

EstadoTarea.java

Enum con los estados posibles: PENDIENTE, EN_PROGRESO, COMPLETADA, CANCELADO.

Subtarea.java

Entidad JPA de la subtarea. Campos: nombre, descripcion, estado (EstadoSubtarea), fecha_inicio, fecha_fin, id_subtarea_orden, prioridad (FK), tarea (FK), usuarios (N:M), marcos (N:M), aplicativos (N:M), auditoria.
Tabla: subtareas.

EstadoSubtarea.java

Enum: PENDIENTE, EN_PROGRESO, COMPLETADA, CANCELADO.

entity/FlujoTareaEntity.java — Tabla: flujos_tarea.

Campos: proyecto (FK), usuario (FK), estado_actual, porcentaje_avance, descripcion, fecha_ampliada.

entity/FlujoSubtareaEntity.java — Tabla: flujos_subtarea.

Misma estructura que flujos_tarea pero asociado a subtarea.

entity/MarcoTareaEntity.java — Tabla: marcos_tarea.

Adjunto de tipo "Marco" vinculado a una tarea.

entity/MarcoSubtareaEntity.java — Tabla: marcos_subtarea.

Adjunto de tipo "Marco" vinculado a una subtarea.

entity/AplicativoTareaEntity.java — Tabla: aplicativos_tarea.

Adjunto de tipo "Aplicativo" vinculado a una tarea.

entity/AplicativoSubtareaEntity.java — Tabla: aplicativos_subtarea.

Adjunto de tipo "Aplicativo" vinculado a una subtarea.

dto/FlujoTareaDTO.java, dto/FlujoSubtareaDTO.java

Objetos de transferencia para flujos de tarea y subtarea.

dto/MarcoTareaDTO.java, dto/MarcoSubtareaDTO.java

Objetos de transferencia para marcos de tarea y subtarea.

dto/AplicativoTareaDTO.java, dto/AplicativoSubtareaDTO.java

Objetos de transferencia para aplicativos de tarea y subtarea.

CAPA DE INFRAESTRUCTURA — infrastructure/src/main/java/.../repositories/repository/

TareaRepository.java

Interface JpaRepository<Tarea, Long>. Metodos clave:

- findByModulo(Modulo): lista tareas por modulo.
- findByTareaIdOrderByIdDesc(Long): tareas por modulo en orden descendente.
- findByModuloIdOrderByTareaOrdenDesc(Long): orden visual Kanban.
- findByModuloIdInOrderByModuloIdAscIdTareaOrdenDesc(List<Long>): multi-modulo.
- incrementarPosiciones / decrementarPosiciones / actualizarPosicionTarea:
queries JPQL @Modifying para reordenar tareas en el tablero.

SubtareaRepository.java

Interface JpaRepository<Subtarea, Long>. Operaciones CRUD de subtareas.

TareaFlujoRepository.java — operaciones sobre flujos_tarea.

SubtareaFlujoRepository.java — operaciones sobre flujos_subtarea.

MarcoTareaRepository.java — operaciones sobre marcos_tarea.

MarcoSubtareaRepository.java — operaciones sobre marcos_subtarea.

AplicativoTareaRepository.java — operaciones sobre aplicativos_tarea.

AplicativoSubtareaRepository.java — operaciones sobre aplicativos_subtarea.

SudoGes — Archivos para programacion de Tareas: Aplicacion y Frontend (cap. 33 cont.)

CAPA DE APLICACION — application/src/main/java/.../application/

CONTROLLERS

TareaController.java

Base URL: /api/modulos/{moduleId}/tareas

Endpoints principales:

- POST / : crearTarea()
- GET / : listarTareas() por modulo
- GET /{tareaid} : obtenerTarea()
- PUT /actualizar/{tareaid} : actualizarTarea()
- DELETE /delete/{tareaid} : eliminarTarea()
- PATCH /actualizarFechaInicio/{id} : actualizarFechaInicio()
- PATCH /actualizarFechaFin/{id} : actualizarFechaFin()
- PATCH /actualizarEstado/{id} : actualizarEstado()
- POST /posicion/{id} : moverTareaAPosicion()
- POST /{id}/usuarios : asignarUsuarios()
- DELETE /{id}/usuarios/{userId} : quitarUsuario()

SubTareaController.java

Base URL: /api/tareas/{tareaid}/subTareas

Misma estructura de endpoints que TareaController aplicada a subtareas.

FlujoTareaController.java — /api/flujos/tareas/*

Endpoints para registrar y consultar flujos de avance de tareas.

FlujoSubtareaController.java — /api/flujos/subtareas/*

Flujos de avance de subtareas.

MarcoTareaController.java — /api/marcos/tareas/*

Listar, subir y descargar archivos de tipo Marco asociados a tareas.

MarcoSubtareaController.java — /api/marcos/subtareas/*

Marcos asociados a subtareas.

AplicativosTareaController.java — /api/aplicativos/tareas/*

Listar, subir y descargar aplicativos de tareas.

AplicativosSubtareaController.java — /api/aplicativos/subtareas/*

Aplicativos de subtareas.

SERVICES

TareaService.java

Logica de negocio central para tareas. Inyecta:

TareaRepository, UsuarioRepository, PrioridadRepository,

ModuloRepository, AuthService, TareaFlujoRepository.

Metodos clave: crearTarea, moverTareaAPosicion, asignarUsuarios, actualizarEstado.

SubTareaService.java — Logica de negocio de subtareas.

FlujoTareaService.java — Gestion de flujos de avance para tareas.

FlujoSubtareaService.java — Flujos de avance para subtareas.

MarcoTareaService.java — Carga/descarga de marcos de tareas.

MarcoSubtareaService.java — Marcos de subtareas.

AplicativoTareaService.java — Carga/descarga de aplicativos de tareas.

AplicativoSubtareaService.java — Aplicativos de subtareas.

SERVICES IMPL (services/impl/)

FlujoTareaServiceImpl.java, FlujoSubtareaServiceImpl.java

MarcoTareaServiceImpl.java, MarcoSubtareaServiceImpl.java

AplicativoTareaServiceImpl.java, AplicativoSubtareaServiceImpl.java

Implementaciones concretas con validacion de tipos de archivo permitidos.

Tipos permitidos en AplicativoTarea: .pdf, .docx, .jpg, .png, .zip, etc.

Tipos bloqueados: .xls, .xlsx, .bat, .exe, .apk (por seguridad).

CAPA FRONTEND — gestor-frontend/src/

COMPONENTES PRINCIPALES

components/KanbanBoard.jsx

Vista principal del tablero Kanban por modulo. Renderiza las tareas como tarjetas arrastrables (react-beautiful-dnd). Gestiona: crear, editar, mover, eliminar tareas, asignar usuarios/prioridad, registrar flujos y navegar a subtareas.

components/KanbanBoardSubtarea.jsx

Vista Kanban de subtareas dentro de una tarea. Misma logica que KanbanBoard pero aplicada al nivel subtarea.

MODALES DE TAREA (components/modal/)

TareaGuardarModal.jsx — Formulario de creacion de tarea (nombre, area, fechas, desc).

TareaEditarModal.jsx — Formulario de edicion de datos de una tarea existente.

TareaVerModal.jsx — Vista de detalle de una tarea (solo lectura).

TareaFlujoModal.jsx — Formulario para registrar flujo de avance de una tarea.

MODALES DE SUBTAREA (components/modal/)

SubTareaGuardarModal.jsx — Creacion de subtarea con validacion de fechas contra tarea padre.

SubTareaEditarModal.jsx — Edicion de subtarea.

SubtareaVerModal.jsx — Vista de detalle de subtarea.

SubtareaFlujoModal.jsx — Flujo de avance de subtarea.

MODALES DE ADJUNTOS PARA TAREA/SUBTAREA

AplicativosTareaModal.jsx — Carga/listado de aplicativos de una tarea.

AplicativosSubtareaModal.jsx — Carga/listado de aplicativos de una subtarea.

MarcoTareaModal.jsx — Carga/listado de marcos de una tarea.

MarcoSubtareaModal.jsx — Marcos de subtarea.

SERVICIOS FRONTEND (services/)

tareaServices/tareaService.js

actualizarFechaInicioTarea, actualizarFechaFinTarea,
deleteTareaService, actualizarNombreTarea.

tareaUsuarioservices/tareaUsuarioService.js

actualizarUsuariosTareas, eliminarUsuarioTareas.

prioridadTareasServices/prioridadTareaService.js

actualizarPrioridadTarea.

subtareaServices/subtareaService.js

actualizarFechaInicio/FinSubtarea, deleteSubtareaService, actualizarNombreSubTarea.

subtareaUsuarioservices/subtareaUsuarioService.js

actualizarUsuariosSubTareas, eliminarUsuarioSubTareas.

prioridadSubtareasServices/prioridadSubtareaService.js
actualizarPrioridadSubtarea.

flujosServices/flujosTareaServices.js — Llamadas al endpoint de flujos de tarea.

flujosServices/flujosSubtareaServices.js — Flujos de subtarea.

aplicativosServices/aplicativosTareaServices.js — Upload/download aplicativos tarea.

aplicativosServices/aplicativosSubtareaServices.js — Aplicativos subtarea.

marcosServices/marcosTareaServices.js — Marcos de tarea.

marcosServices/marcosSubtareaServices.js — Marcos de subtarea.